EXPLOITS UNIVERSITY

BSc INFORMATION COMMUNICATION AND TECHNOLOGY

BICT 3202 · OBJECT-ORIENTED ANALYSIS AND DESIGN

WEEK 15

Complete OOAD Case Study, Model Integration and CASE Tool Implementation

PROGRAMME	BSc Information Communication and Technology
COURSE CODE / YEAR	BICT 3202 · Year 3
LECTURER	Francis Fweta — ICT Lecturer
DELIVERY	2h Lecture · 1h Tutorial · 1h Practical · 10h Independent Learning

Prepared by Francis Fweta · Exploits University · Week 15 of 16

Contents

1. Week 15 Theme 4
2. Learning Outcomes 4
PART A — WHAT HAVE WE BEEN LEARNING? 5
3. Looking Back at the Entire Module 5
4. Why Integration Matters 5
PART B — THE COMPLETE CASE STUDY 6
5. Case Study: University Online Course Registration System 6
PART C — REQUIREMENTS 7
6. Functional and Non-Functional Requirements 7
PART D — IDENTIFYING ACTORS 7
7. What Is an Actor? 7
PART E — USE-CASE MODEL 8
8. Student and Administrator Use Cases 8
PART F — FULL USE-CASE SPECIFICATION 9
9. Use Case: Register for Course 9
PART G — IDENTIFYING OBJECTS 9
10. Candidate Classes 9
PART H — CLASS MODEL 10
11. The Design Classes 10
PART I — ASSOCIATIONS 11
12. Student, Course and Registration 11
PART J — INHERITANCE 12
13. Should We Use Inheritance? 12
PART K — SEQUENCE DIAGRAM 13
14. Register Course Interaction 13
PART L — STATE MACHINE 13
15. Course Lifecycle 13
PART M — ACTIVITY DIAGRAM 14
16. Registration Workflow 14

PART N — COMPONENT DESIGN 15
17. From Classes to Components 15
PART O — MODEL TRACEABILITY 16
18. Requirement -> Implementation 16
PART P — CONSISTENCY CHECK 16
19. Performing a Model Audit 16
PART Q — FULL ANALYSIS DOCUMENTATION 17
20. What Should a Complete OOAD Document Contain? 17
PART R — CASE TOOLS 17
21. What Is a CASE Tool? 17
PART S & T — CASE TOOL PRACTICAL AND THE RIGHT MINDSET 18
22. Practical Demonstration and How to Think 18
PART U — COMMON CASE-TOOL MISTAKES 18
23. Four Classic Mistakes 18
PART V — COMPLETE INTEGRATED MODEL 18
24. The Big Picture 18
PART W — TUTORIAL SESSION (1 HOUR) 19
25. Tutorial: Online Library Management System 19
PART X — PRACTICAL SESSION (1 HOUR) 19
26. Practical Deliverables and Quality Requirements 19
PART Y — INDEPENDENT LEARNING (10 HOURS) 20
27. Major Week 15 Assignment — Ten Deliverables 20
PART Z — ASSESSMENT RUBRIC (100 MARKS) 20
28. Important Distinctions Students Must Know 21
29. Examination-Style Questions 21
30. Week 15 Summary 22
31. References and Further Reading 22

1. Week 15 Theme

Week 15 is the **integration and application week**. Students now take the concepts developed from Weeks 1–14 and apply them to a complete system. The emphasis is not simply on drawing UML diagrams, but on demonstrating that the diagrams **tell a consistent story and can guide implementation**.

The central question is: *"If I am given a real software problem today, can I use everything I have learned to analyse it, model it, design it and communicate the design to another developer?"* By now, use-case, class, sequence, activity, state and component diagrams should no longer feel like separate examination topics — they are different views of one software system.

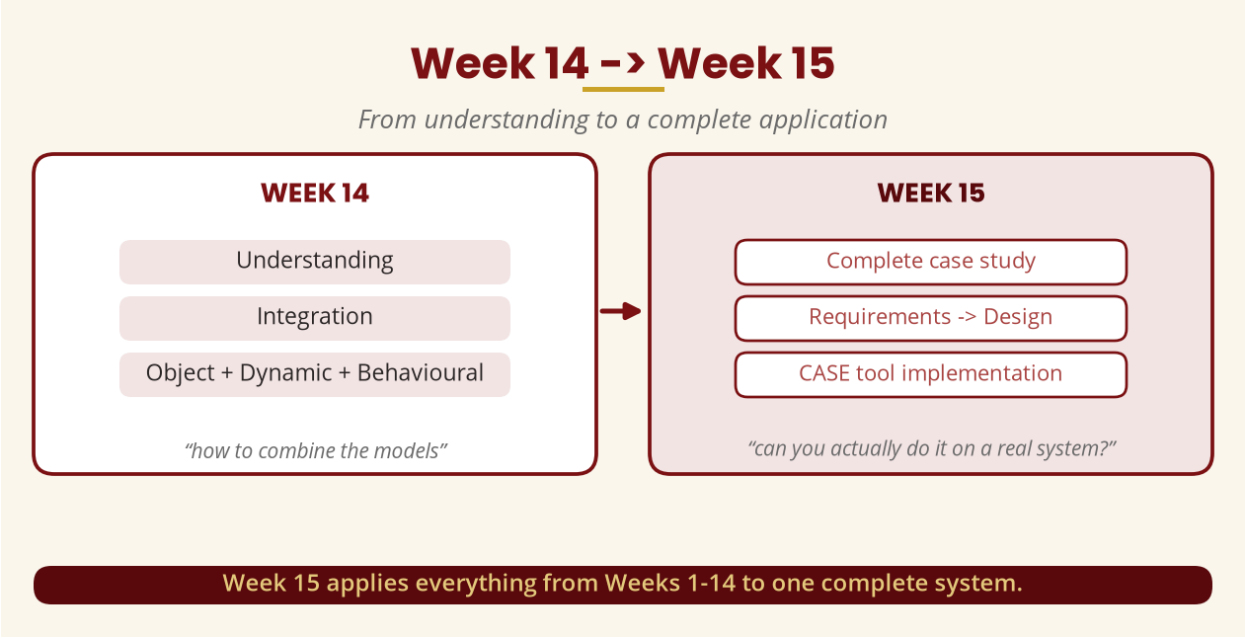


Figure 1 — Week 14 -> Week 15

2. Learning Outcomes

By the end of this week, a student should be able to:

- Analyse a complete software problem using object-oriented techniques.
- Identify stakeholders and system actors; extract functional and non-functional requirements.
- Identify appropriate use cases and construct a use-case model.
- Identify candidate classes and develop a conceptual/domain class model.
- Refine the analysis model into a design model; identify class responsibilities.
- Define attributes, operations, associations, inheritance and composition.
- Develop interaction, activity and state-machine models for important behaviours.
- Develop a component-level view; check consistency between UML models.
- Document a complete OOAD solution and use a CASE/UML tool to organise models.
- Explain how the final design could be implemented.

PART A — WHAT HAVE WE BEEN LEARNING?

3. Looking Back at the Entire Module

The OOAD process can be simplified as a journey: Business Problem -> Requirements -> Use Cases -> Object Identification -> Class Model -> Dynamic Model -> Behavioural Model -> Analysis Documentation -> Design -> Architecture/Components -> Integrated Design -> Implementation. The single most important word is **INTEGRATION**.



Figure 2 — The OOAD journey

4. Why Integration Matters

If the use-case diagram shows `Student -> Register Course` but the class diagram has no `Registration` class, the models are incomplete. If the class diagram says `Course.checkCapacity()` but the sequence diagram calls `verifySeats()`, there is a consistency problem. If the activity diagram says "Check payment" but no payment-related class, operation or component exists anywhere, something is missing. Therefore: **a good OOAD model must be internally consistent.**

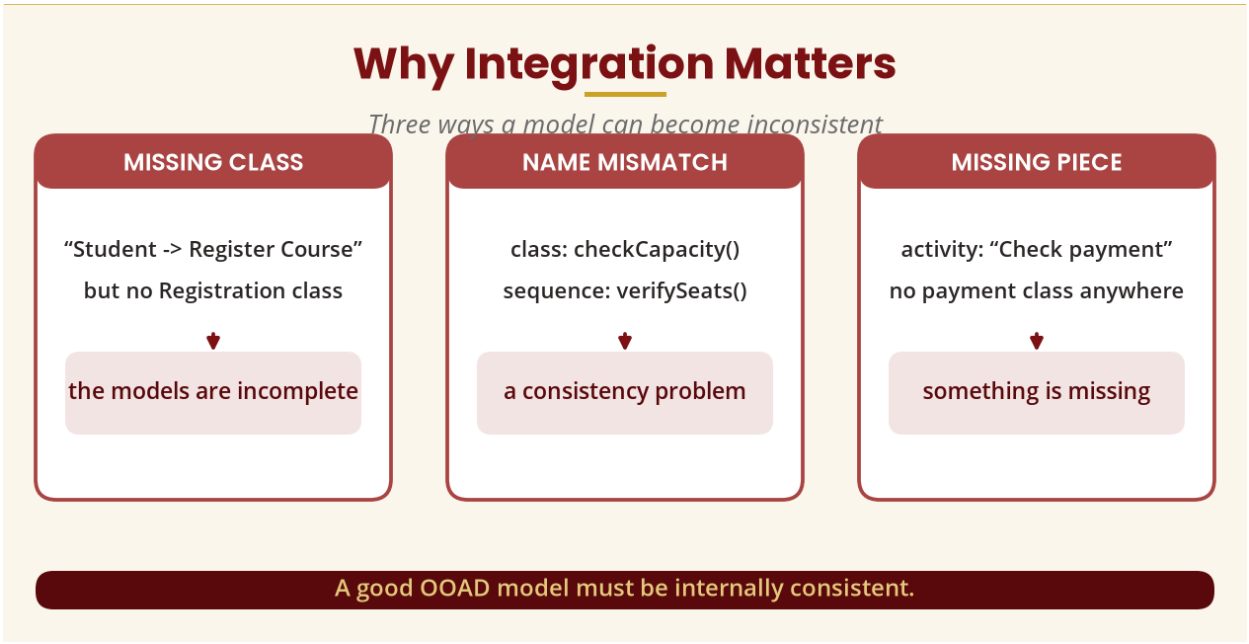


Figure 3 — Why integration matters

PART B — THE COMPLETE CASE STUDY

5. Case Study: University Online Course Registration System

The university wants students to register for courses online instead of filling out paper forms. Today it suffers from long queues, lost forms, duplicate registrations, students registering without prerequisites, courses exceeding capacity, delayed confirmation and poor reporting.

Business goal: students should search for courses, check eligibility, register and receive confirmation electronically; administrators should create/modify courses, set capacity, view registrations, close registration and generate reports.

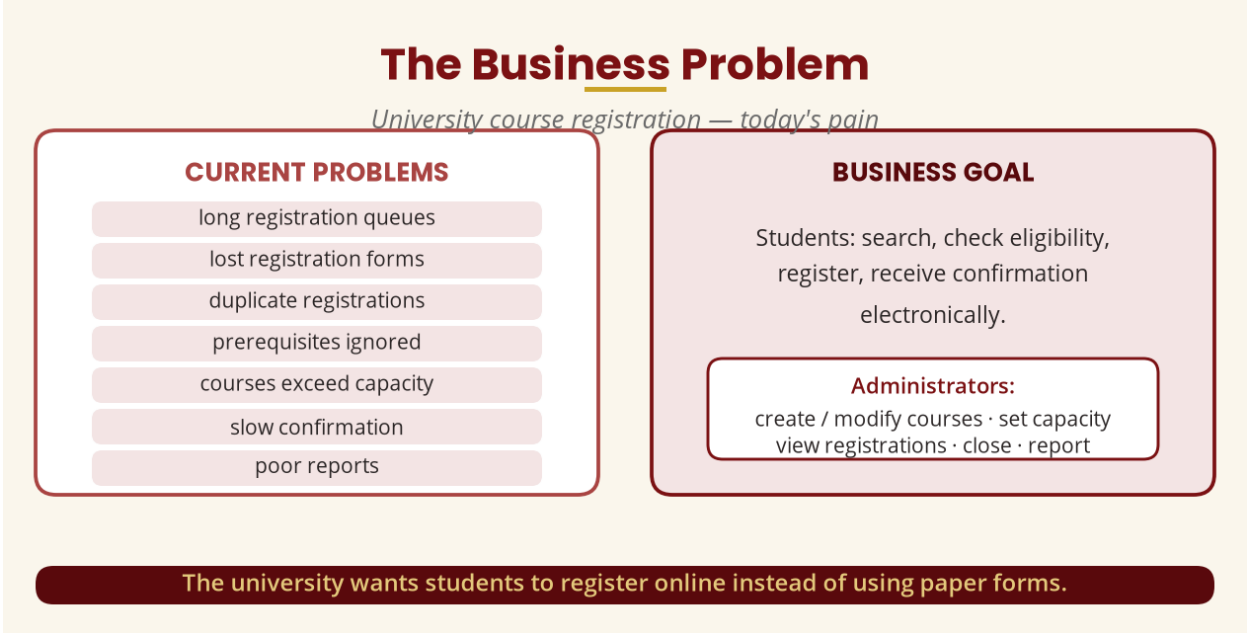


Figure 4 — The business problem and goal

PART C — REQUIREMENTS

6. Functional and Non-Functional Requirements

A **functional requirement** describes something the system must do. For this system: FR1 student login, FR2 view courses, FR3 search courses, FR4 check eligibility, FR5 register, FR6 prevent duplicate registration, FR7 check capacity, FR8 drop course, FR9 confirmation, FR10 administration.

A **non-functional requirement** describes how well the system operates: security (authorised access only), performance (acceptable response time), availability (up during registration periods), usability (no extensive training) and maintainability (future changes without a rewrite).

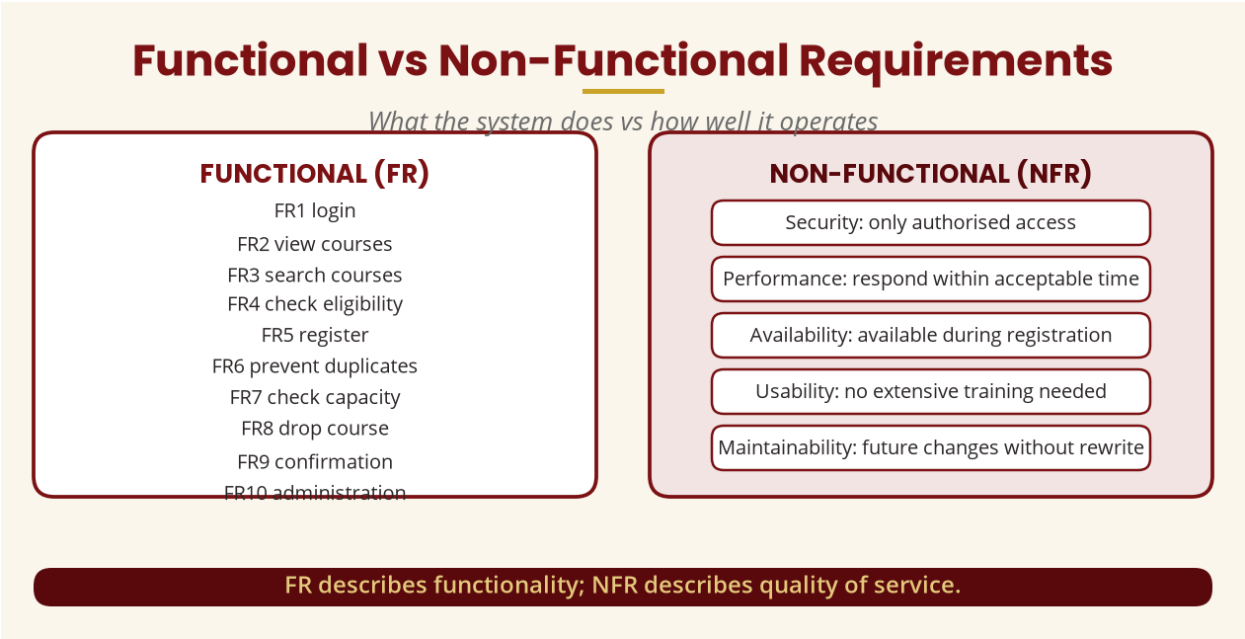


Figure 5 — Functional vs non-functional requirements

PART D — IDENTIFYING ACTORS

7. What Is an Actor?

An **actor** is an external role that interacts with the system — not necessarily a human. Actors can be a student, administrator, lecturer, payment service, email service or another software system. For our system we identify: **Student, Administrator, Lecturer, Notification Service** and (potentially) **Payment Gateway**.

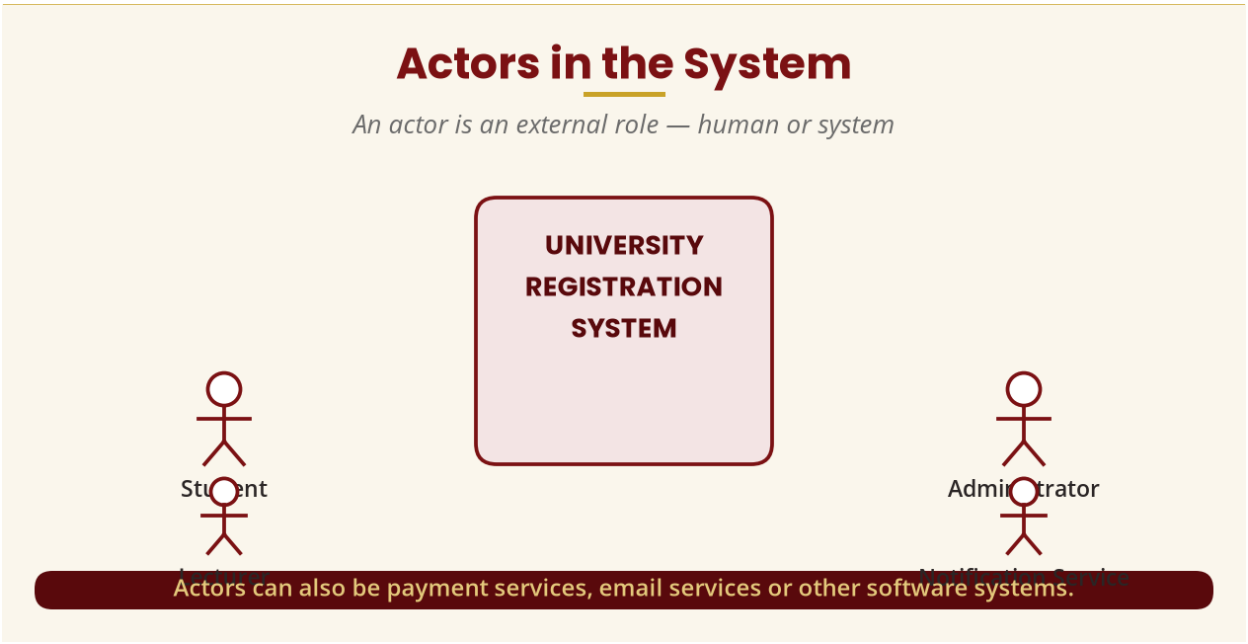


Figure 6 — Actors in the system

PART E — USE-CASE MODEL

8. Student and Administrator Use Cases

A **student** might: Login, View Courses, Search Courses, Register Course, Drop Course, View Registration, Download Registration Confirmation. An **administrator** might: Login, Create Course, Update Course, Delete Course, Set Course Capacity, View Registrations, Close Registration, Generate Report.

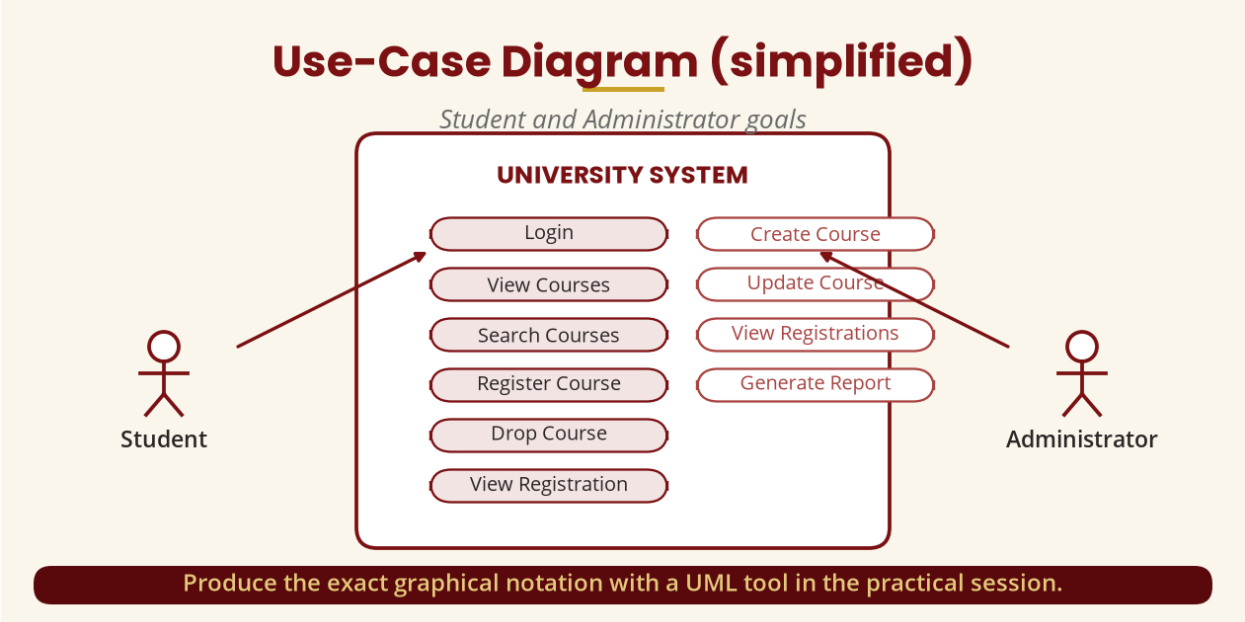


Figure 7 — Simplified use-case diagram

PART F — FULL USE-CASE SPECIFICATION

9. Use Case: Register for Course

Name: Register for Course · **Primary Actor:** Student · **Goal:** register for an available course · **Preconditions:** authenticated, registration period open, account active.

Main success scenario: (1) the student selects an open course; (2) the system checks prerequisites and rejects an active duplicate; (3) `Course.reserveSeat()` atomically claims a seat; (4) the system creates and saves the Registration in the same transaction; (5) after the transaction commits, the system sends confirmation and displays the result.

Alternative flows: if the prerequisite fails, a duplicate exists or no seat can be reserved, the system rejects the request without creating or saving a new Registration. If persistence fails after a seat claim, the transaction rolls the claim back. These alternatives are shown in the sequence/activity models.

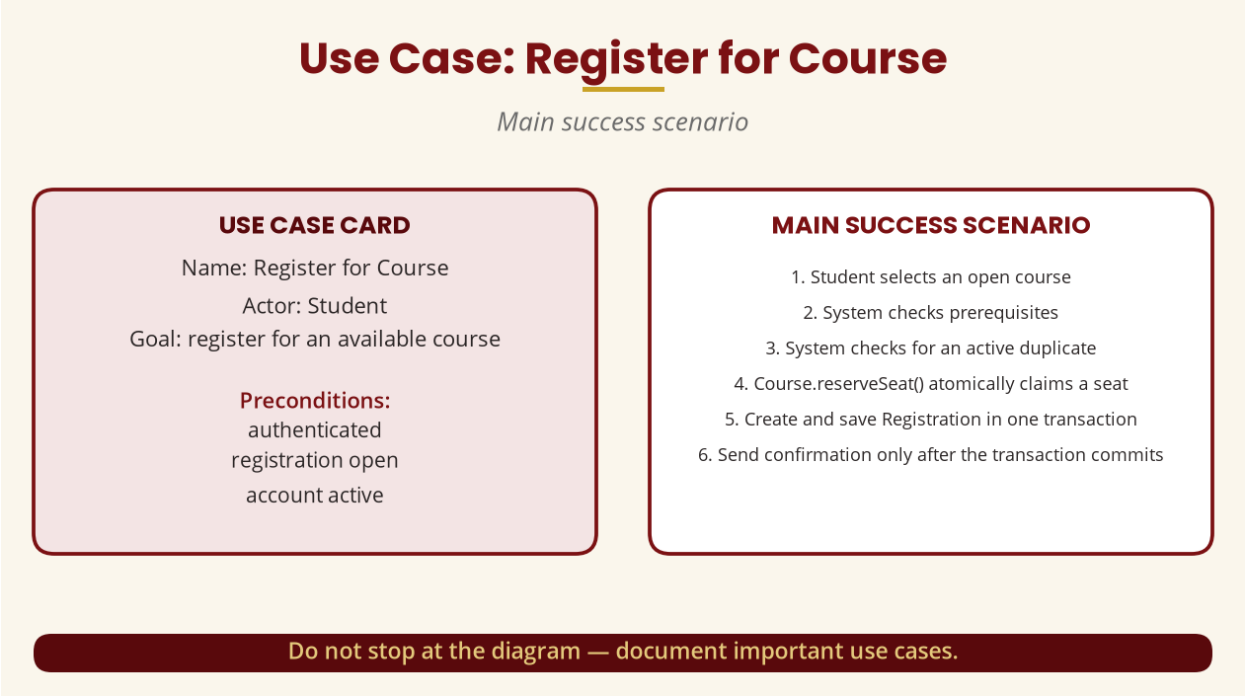


Figure 8 — Use-case specification

PART G — IDENTIFYING OBJECTS

10. Candidate Classes

From the requirements we identify **domain classes** — Student, Course, Registration, Programme, a PrerequisiteRule where policy is complex, Administrator and Lecturer — and **technical/application classes** — RegistrationController, RegistrationService, StudentRepository, CourseRepository, RegistrationRepository, NotificationService and AuthenticationService. Domain classes represent the problem domain; technical classes support implementation. **Do not confuse these.**

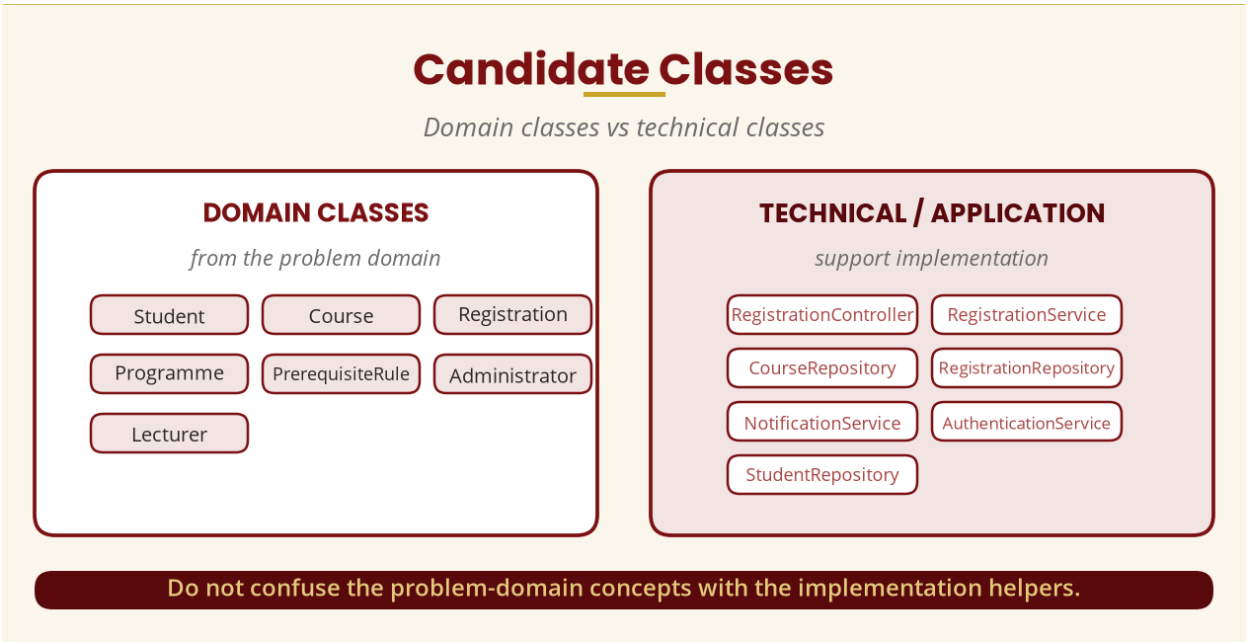


Figure 9 — Domain vs technical classes

PART H — CLASS MODEL

11. The Design Classes

Refined design classes include: `Student` (studentId, name, email, programme, completedCourses; isEligibleFor(course), dropCourse()), `Course` (courseCode, courseName, credits, capacity, enrolledCount, status; prerequisite self-association, hasCapacity(), reserveSeat(), releaseSeat()), `Registration` (registrationId, registrationDate, status; create(student, course), confirm(), cancel()), `RegistrationService` (registerStudent(), dropStudent(), checkEligibility(), checkDuplicate()) and `RegistrationRepository` (save(), existsActiveFor(), findByStudent(), findByCourse()). A complex prerequisite policy can have its own `PrerequisiteRule` class. Notice the movement from "register a course" to a detailed design: **Controller -> Service -> Domain objects -> Repository.** The model should state that each Registration connects exactly one Student and one Course. A database uniqueness constraint or equivalent transaction must prevent concurrent duplicate active registrations.

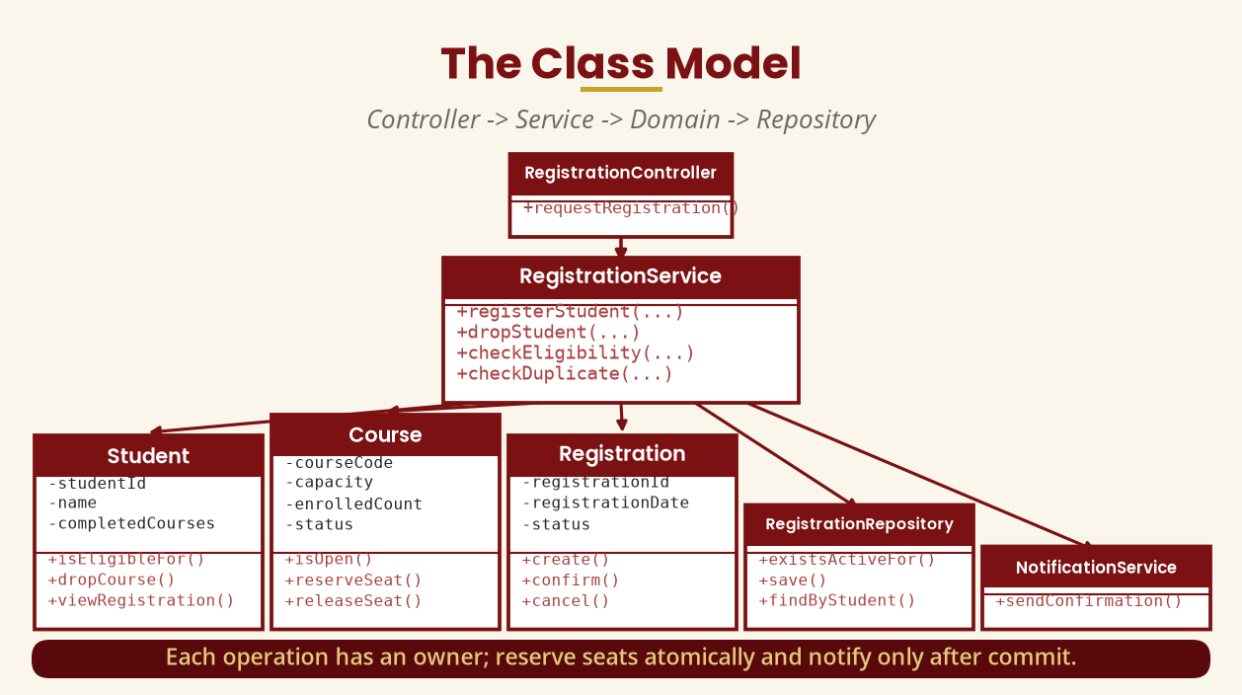


Figure 10 — The class model

PART I — ASSOCIATIONS

12. Student, Course and Registration

A student can have many registrations: `Student 1 — 0..* Registration` — one student may have zero or many registration records. A course can have many registrations: `Course 1 — 0..* Registration`. The Student-to-Course completed-course association supports prerequisite checks; Course can also have a recursive prerequisite association with explicit target/prerequisite roles. Use a `PrerequisiteRule` class if policy has compound conditions. Combined, these relationships are more meaningful than simply drawing lines between classes — **students must understand the multiplicity.**

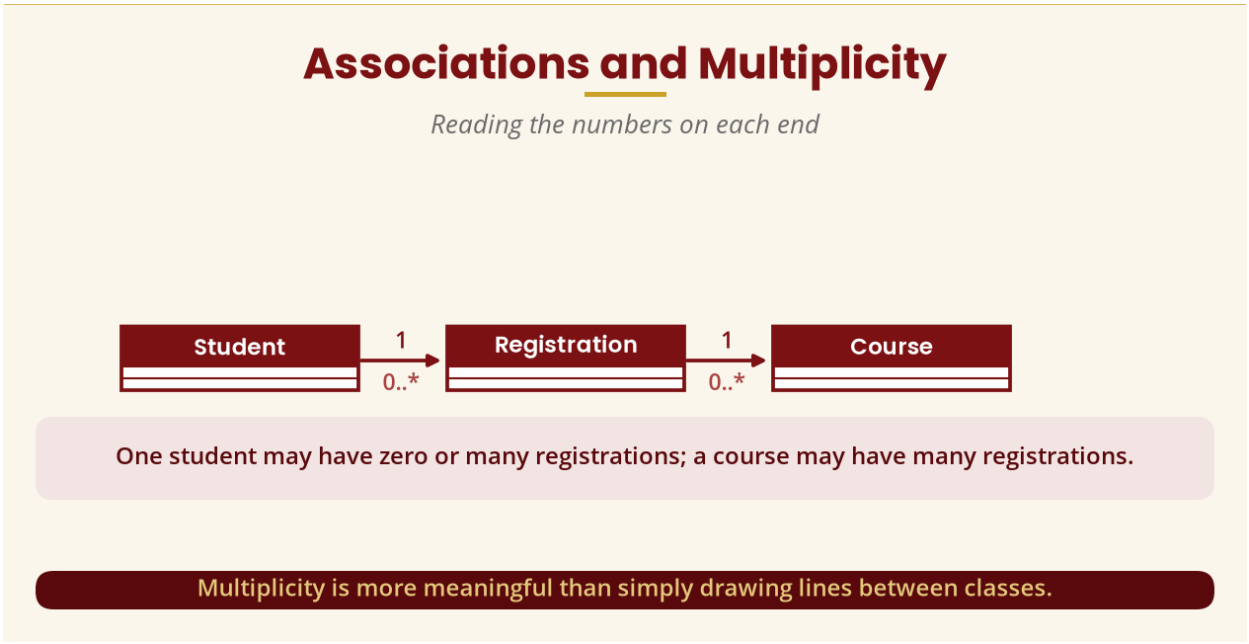


Figure 11 — Associations and multiplicity

PART J — INHERITANCE

13. Should We Use Inheritance?

Student, Administrator and Lecturer all share a user ID, name, email and authentication behaviour, but have different responsibilities — so a general `User` abstraction with `Student`, `Administrator` and `Lecturer` as specialisations may be reasonable. However, **do not use inheritance simply because UML allows it** — it must be genuinely appropriate.

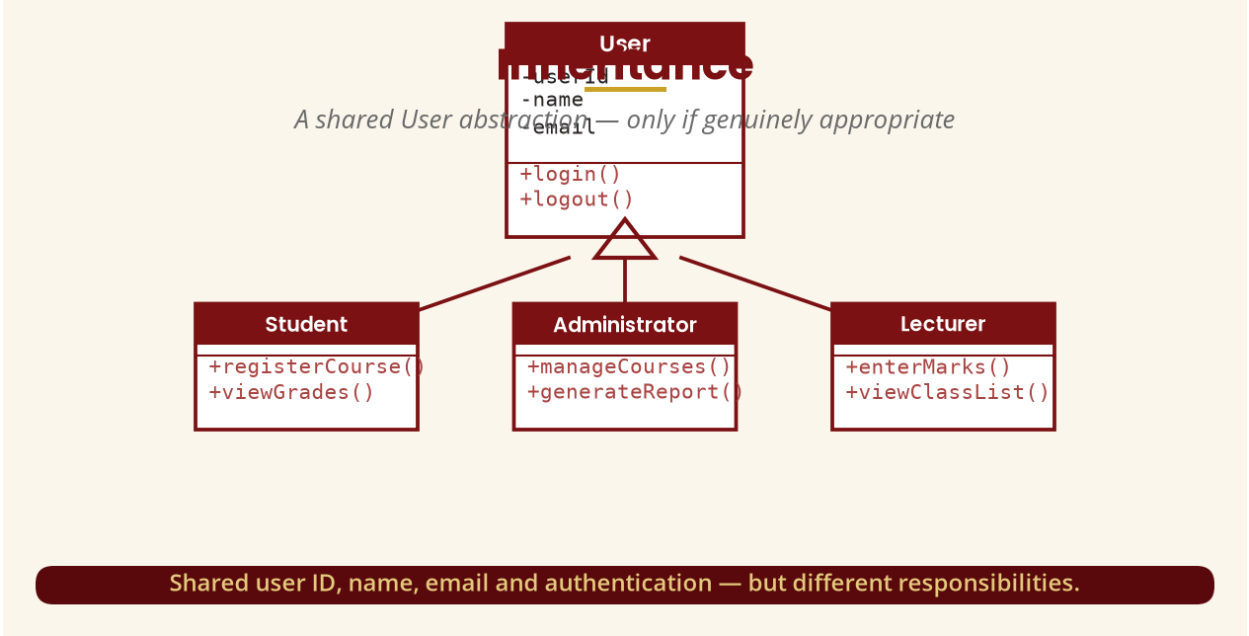


Figure 12 — Inheritance: a shared User abstraction

PART K — SEQUENCE DIAGRAM

14. Register Course Interaction

Student sends `requestRegistration(courseCode)` to `RegistrationController`, which calls `RegistrationService.registerStudent()`. The service checks eligibility, asks `RegistrationRepository.existsActiveFor()` to reject duplicates, then calls `Course.reserveSeat()`. Only when the guarded checks succeed does it create and save `Registration` and send confirmation through `NotificationService` only after the registration transaction commits. An `alt` fragment shows the rejection path. **reserveSeat() must be atomic:** a separate capacity check followed by a later update can overbook the final seat under concurrent requests. Enforce duplicate prevention atomically too: an existence query alone can race, so use a unique active-registration constraint or equivalent transaction. Coordinate seat reservation and save in one transaction; if persistence fails, roll the seat back. The class diagram shows *what exists*; the sequence diagram shows *how those classes cooperate*.

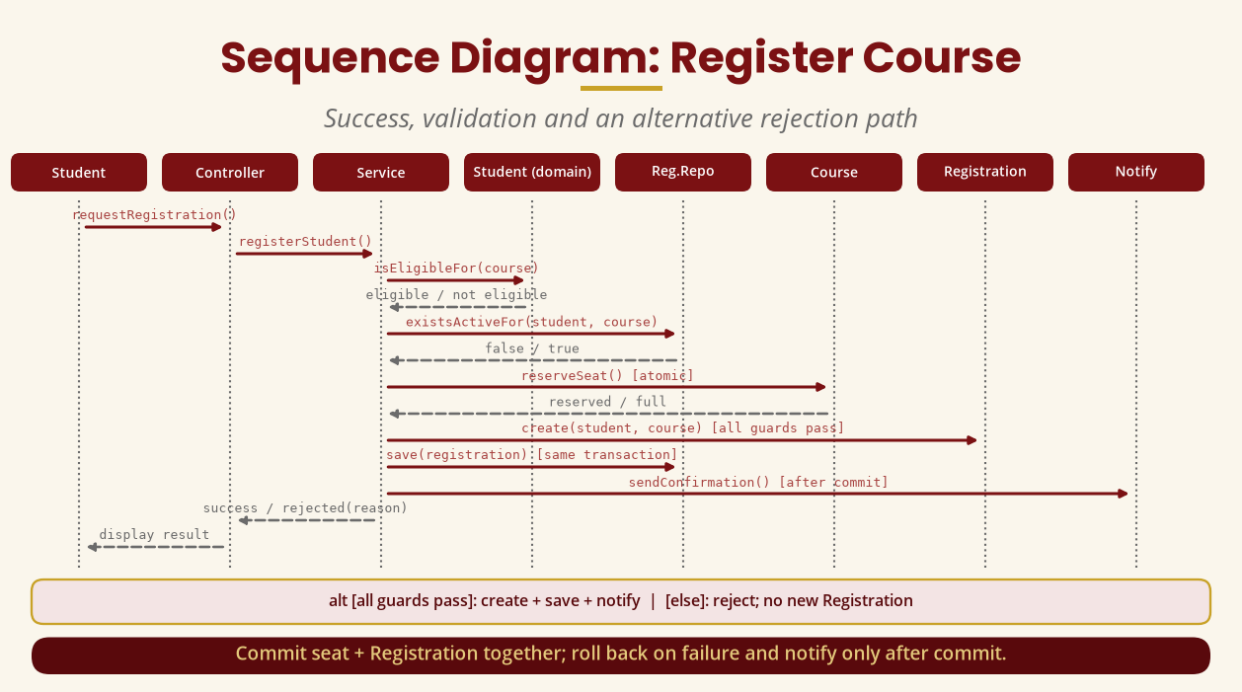


Figure 13 — Sequence diagram: Register Course

PART L — STATE MACHINE

15. Course Lifecycle

A course moves through `Created -> Available -> Full -> Closed`. Transitions: **Created -> Available** when an authorised administrator opens registration; **Available -> Full** when an atomic `reserveSeat()` succeeds for the final seat; **Full -> Available** when a seat is released; and **Available/Full -> Closed** when the registration period ends. Keep the guard and capacity invariant in view: `enrolledCount <= capacity`.

We can put `status : CourseStatus` in the class, but that does not explain *what causes the status to change*. The state machine supplies that dynamic information and should agree with the sequence diagram.

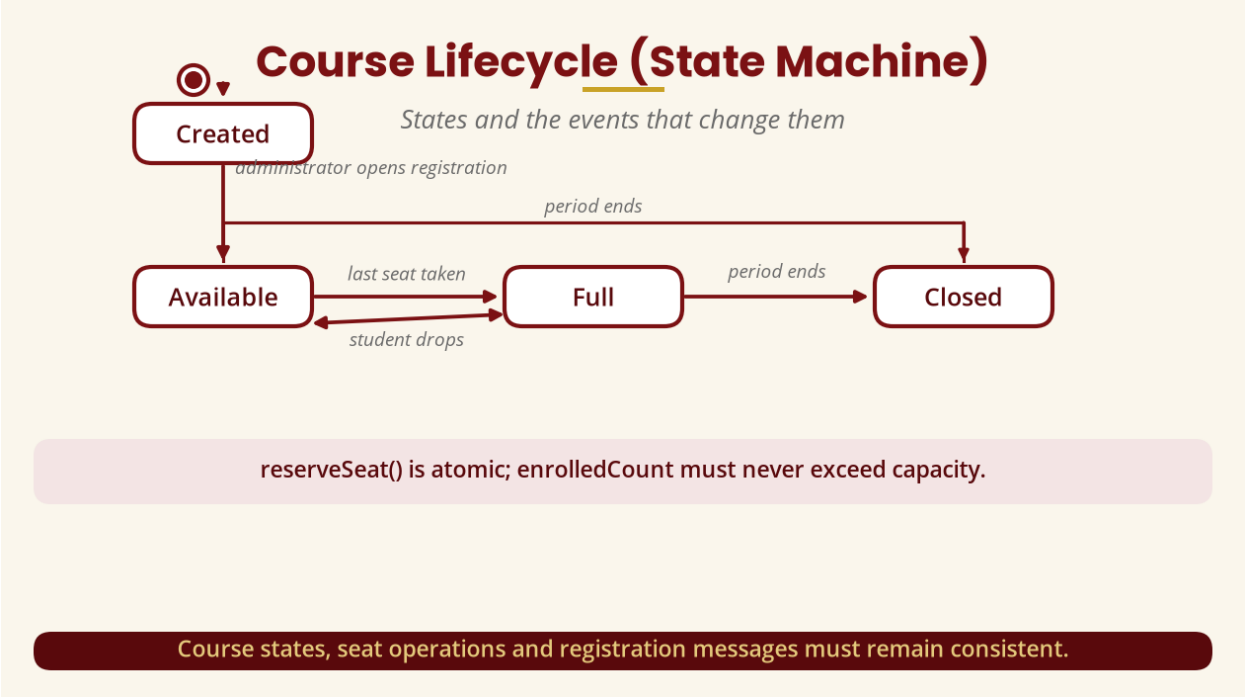


Figure 14 — Course lifecycle state machine

PART M — ACTIVITY DIAGRAM

16. Registration Workflow

The workflow: Start -> authenticate -> select open course -> check prerequisites -> *[eligible?]* No -> reject / Yes -> check for duplicate -> *[already registered?]* Yes -> reject / No -> atomically reserve a seat -> *[seat reserved?]* No -> report full / Yes -> create and save Registration in the seat transaction -> commit -> send confirmation -> End. The activity diagram describes workflow decisions; the sequence diagram assigns those actions to objects. A failed guard must not create or save a new Registration.

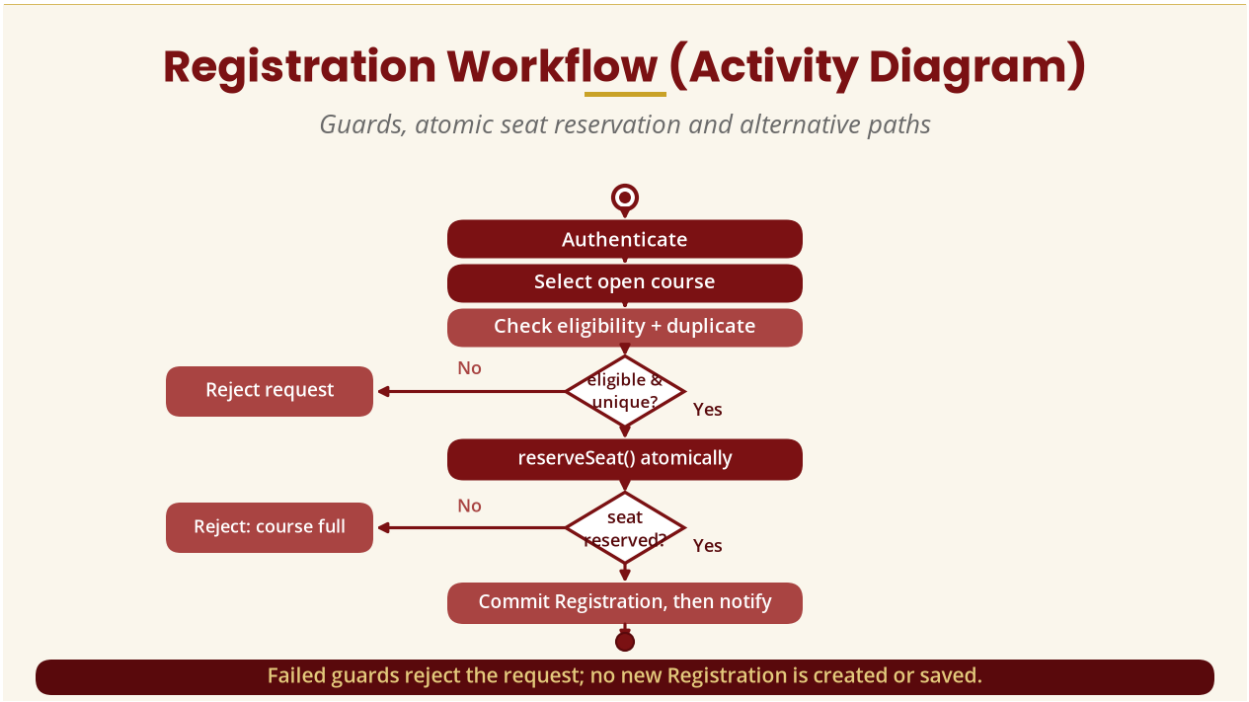


Figure 15 — Registration workflow activity diagram

PART N — COMPONENT DESIGN

17. From Classes to Components

At design level we group related functionality into components — Student Management, Course Registration, Authentication and Notification — and organise the system into layers: **Presentation** (web/mobile interface), **Application** (controllers/services), **Domain** (Student/Course/Registration) and **Persistence** (repositories/database). This connects directly with the architectural concepts covered earlier in the module.

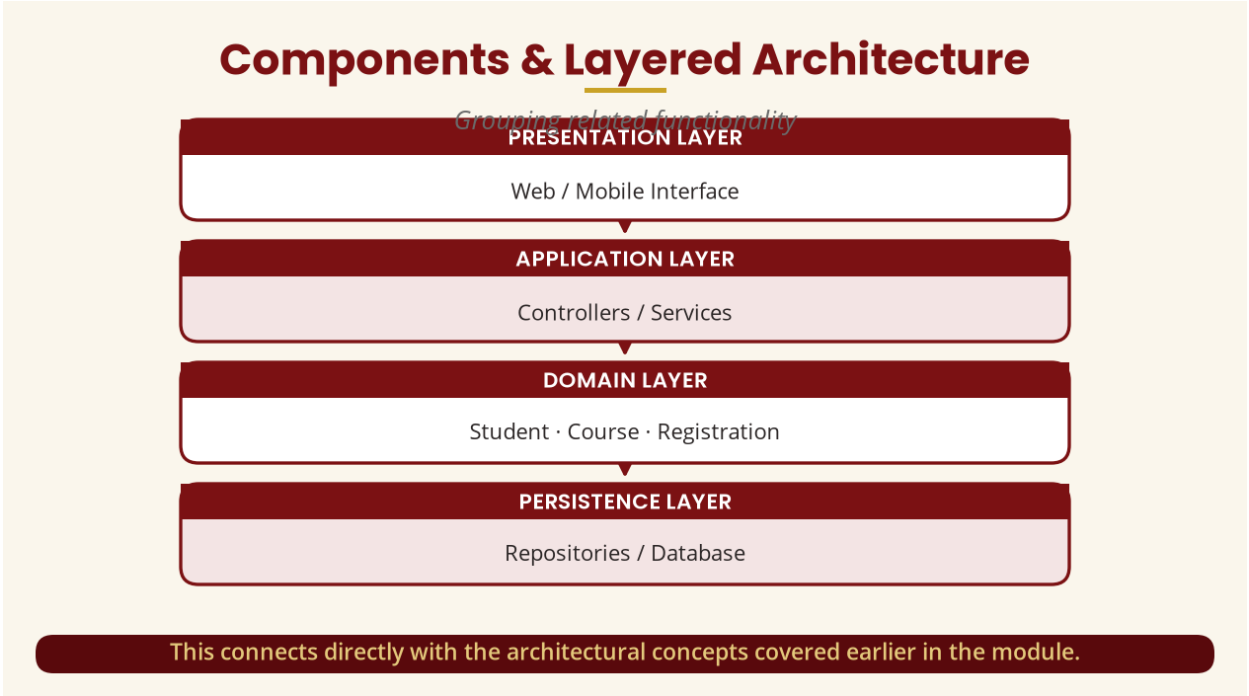


Figure 16 — Components and layered architecture

PART O — MODEL TRACEABILITY

18. Requirement -> Implementation

Trace FR5 (register an eligible student) from the `Register Course` use case to `RegistrationService.registerStudent()`, the `Registration` class and its save operation. Trace FR6 (prevent duplicates) to `existsActiveFor()` and its guarded rejection path; trace FR7 (capacity) to atomic `Course.reserveSeat()` and the Available/Full state transition. NFRs such as role-based access should map to the authentication boundary and test evidence. This is stronger than a one-way diagram list: each requirement has a model/design element and a way to validate it.

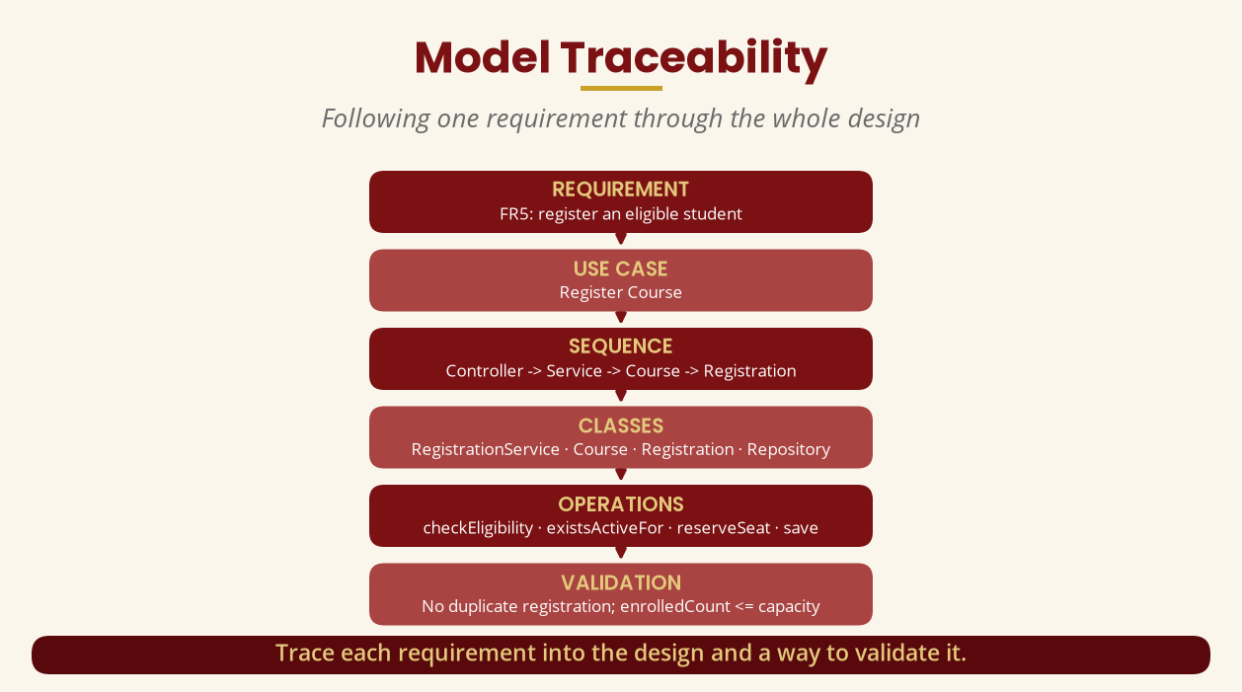


Figure 17 — Model traceability

PART P — CONSISTENCY CHECK

19. Performing a Model Audit

- **Use Case <-> Class Model:** does every major use case have supporting classes?
- **Class Model <-> Sequence Diagram:** does every important message correspond to an operation?
- **Sequence <-> State Model:** do messages cause appropriate state transitions?
- **Activity <-> Sequence:** does the workflow correspond to the interaction?
- **Class <-> Component:** are classes sensibly allocated to components?
- **Requirements <-> Design:** can each important requirement be traced into the design?

Example defect: if the activity diagram says "Check Payment" but the class diagram contains no payment-related class, ask "how is payment being checked?" — the solution may require introducing `PaymentService`, `Payment` and `PaymentGateway`. This is model validation.

PART Q — FULL ANALYSIS DOCUMENTATION

20. What Should a Complete OOAD Document Contain?

- Introduction (background, problem statement, purpose).
- Requirements (functional and non-functional).
- Actors and use cases (with the use-case diagram).
- Domain/class model (important classes).
- Interaction models (sequence/collaboration diagrams).
- Dynamic model (state diagrams) and activity model (workflows).
- Design (refined classes and responsibilities) and architecture (layers/components).
- Model validation (consistency) and conclusion.

PART R — CASE TOOLS

21. What Is a CASE Tool?

CASE means Computer-Aided Software Engineering. A CASE tool provides software support for requirements analysis, modelling, UML diagram creation, documentation, project management and code-related development activities. For this course the important part is **using a tool to create and manage UML models**.

Imagine drawing 15 UML diagrams manually and then discovering "the class name is wrong in six diagrams" — you would have to correct everything by hand. A modelling tool makes model management much easier through diagram editing, model repositories, documentation, consistency support, project organisation and sometimes code engineering.

Examples: Enterprise Architect, Visual Paradigm, StarUML, Modelio, and diagrams.net/draw.io. The exact tool matters less than understanding the modelling principles; the course outline specifically mentions a **CASE Tool: Select Enterprise Demonstration**.

PART S & T — CASE TOOL PRACTICAL AND THE RIGHT MINDSET

22. Practical Demonstration and How to Think

In the practical session, the lecturer demonstrates creating: (1) the project "University Course Registration"; (2) packages for Requirements, Analysis and Design; (3) the use-case diagram; (4) the class diagram; (5) the sequence diagram for Register Course; (6) the activity diagram for the Registration Process; (7) the state diagram for the Course Lifecycle; and (8) the component diagram for the System Architecture.

The tool does not do the thinking. Knowing Enterprise Architect is like knowing Microsoft Word — knowing Word does not mean knowing what to write, and knowing a CASE tool does not mean understanding object-oriented analysis and design. The student must first understand `Problem -> Model -> Design`, then use the tool to represent that model.

PART U — COMMON CASE-TOOL MISTAKES

23. Four Classic Mistakes

- **Drawing diagrams without requirements** — opening the tool and immediately drawing classes; the correct order is requirements -> understand system -> actors -> use cases -> classes -> diagrams.
- **Creating too many classes** — StudentManager, StudentProcessor, StudentHandler, StudentHelper...; always ask: what responsibility does this class have?
- **Copying diagrams from the internet** — UML notation is standard, but the model must represent your system.
- **Inconsistent naming** — class "Registration" but sequence "Enrollment" and database "CourseSignup"; establish a consistent vocabulary.

PART V — COMPLETE INTEGRATED MODEL

24. The Big Picture

Student and Administrator act on Use Cases, which feed the Analysis Model; the analysis branches into the Class Model (classes), Dynamic Model (states) and Behavioural Model (interactions); these converge into the Design Model, then Architecture, then Components, then Implementation. **This is the complete OOAD journey.**

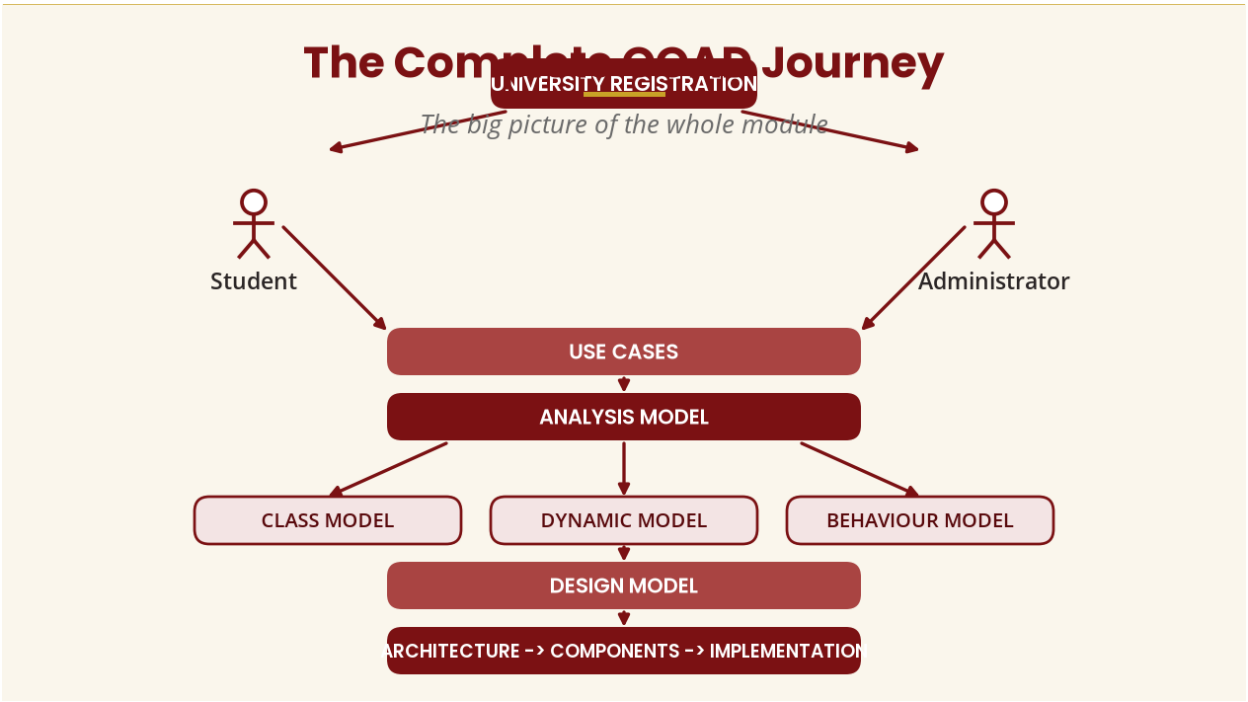


Figure 18 — The complete OOAD journey

PART W — TUTORIAL SESSION (1 HOUR)

25. Tutorial: Online Library Management System

A college wants an Online Library Management System. **Students** can search, borrow, return and view borrowed books. **Librarians** can add books, remove books, register members and view overdue books.

Tasks: (1) identify actors, use cases and candidate classes; (2) create the use-case diagram; (3) create the class diagram; (4) select “Borrow Book” and create a sequence diagram; (5) create an activity diagram for borrowing; (6) create a state diagram for Book Copy (Available -> Borrowed -> Returned -> Available); (7) explain how the five diagrams relate.

PART X — PRACTICAL SESSION (1 HOUR)

26. Practical Deliverables and Quality Requirements

Using a UML/CASE tool, implement the University Registration case study with six diagrams: use-case, class, sequence, activity, state-machine and component.

Quality requirements: naming consistency (Registration should not randomly become Enrollment); every relationship must have a reason; multiplicity (1, 0..1, 0..*, 1..*) used where appropriate; sequence messages must correspond to class operations; state transitions triggered by meaningful events; every major requirement represented somewhere in the model.

PART Y — INDEPENDENT LEARNING (10 HOURS)

27. Major Week 15 Assignment — Ten Deliverables

Select one real-world information system (banking, hospital management, e-commerce, library, hotel reservation, university registration, mobile money or airline reservation) and produce:

(1) a problem definition (background, problem statement, objectives, scope); (2) at least 10 functional and 5 non-functional requirements; (3) actors and major use cases with relationships; (4) a complete use-case diagram; (5) a class model of at least 10 meaningful classes with attributes, operations, visibility and relationships; (6) at least two sequence diagrams for important use cases; (7) at least one activity diagram; (8) a state diagram for a state-dependent object (Order, Ticket, Account, Book, Course, Application or Payment); (9) a component diagram (Authentication, Student Management, Course Management, Registration, Notification, Reporting, Database); and (10) an integration analysis answering how the use-case model relates to the class model, how the sequence relates to operations, how the state relates to behaviour, how the activity relates to the workflow, how the component relates to the classes, and how the complete model supports the requirements.

PART Z — ASSESSMENT RUBRIC (100 MARKS)

Area	Marks
Problem definition	5
Requirements	10
Use-case model	10
Class model	15
Sequence diagrams	15
Activity diagram	10
State model	10
Component/design model	10
Model consistency/integration	10
Documentation/presentation	5
Total	100

28. Important Distinctions Students Must Know

Concept	What it answers
Requirements	What does the stakeholder need?
Use case	What goal does an actor accomplish?
Class diagram	What structural elements exist?
Sequence diagram	How do objects communicate over time?
Activity diagram	How does a workflow proceed?
State diagram	How does one object change state?
Component diagram	How is functionality organised into components?
Design model	How should the system be structured for implementation?
CASE tool	What software can help create/manage the models?

At Year 3 level, "a class diagram shows the system and a sequence diagram shows interaction" is too vague. The expected answer: *"A class diagram models the structural/static aspects of the system by representing classes, attributes, operations and relationships, whereas a sequence diagram models a particular interaction scenario by showing the ordered exchange of messages between participating objects."*

29. Examination-Style Questions

Question 1 (20 marks). Explain the purpose of CASE tools in object-oriented analysis and design, discussing at least five ways in which CASE tools support software development.

Question 2 (25 marks). Using an Online Library Management System, explain how requirements are transformed into use cases, classes, interactions and a final object-oriented design.

Question 3 (25 marks). Explain why UML diagrams should not be developed independently, demonstrating how use-case, class, sequence, activity and state-machine diagrams describe different aspects of the same system.

Question 4 (30 marks). A university wants an online course registration system. Develop an OOAD solution including (a) actors, (b) use cases, (c) candidate classes, (d) class relationships, (e) major operations, (f) the registration interaction sequence, (g) course states, (h) the registration workflow, (i) components, and (j) an explanation of how the models remain consistent.

30. Week 15 Summary

THE MOST IMPORTANT LESSON

OOAD is not about drawing many UML diagrams. It is about building a coherent model of a software system where requirements, use cases, objects, classes, interactions, behaviour and design decisions all support one another. A student who can take a new problem from **Requirement -> Use Case -> Class -> Interaction -> Behaviour -> Design -> Components -> Integrated UML Documentation** has genuinely understood Object-Oriented Analysis and Design — and that is exactly what Week 15 is designed to test.

31. References and Further Reading

Primary standards

- Object Management Group (OMG). *Unified Modeling Language (UML), Version 2.5.1.* OMG — formal UML specification (adopted December 2017), organising UML into structural, behavioural and supplementary concepts.
- OMG. *Introduction to OMG Specifications.* Overview of UML diagram categories.

Process frameworks

- IBM. *Rational Unified Process (RUP)* — four phases (Inception, Elaboration, Construction, Transition) with iterations as planned intervals.
- IBM. *RUP Best Practices* — Inception establishes the business case and scope; Elaboration analyses the domain, establishes the architectural foundation and addresses high-risk areas.

Prescribed and recommended texts

- Mach, E. (2019). *Object Oriented Analysis & Design Cookbook: Introduction to Practical System Modeling.*
- Reddy, V., & Korra, S. (2018). *Object-Oriented Analysis and Design Using UML.*
- The Art of Service. (2021). *Object Oriented Analysis and Design: A Complete Guide.*
- Wixom, B., & Dennis, A. (2018). *Systems Analysis and Design.*
- Blokdyk, G. (2021). *Object-Oriented Analysis and Design Standard Requirements.*
- Wixom, B., & Dennis, A. (2020). *Systems Analysis and Design: An Object-Oriented Approach with UML.*